CONTRACT WOLF

Security Assessment

FUSDExchange V3

Verified on 03/29/2026

SUMMARY

Project	CHAIN	METHODOLOGY
FUSDExchange V3	Binance Smart Chain	Manual & Automatic Analysis
FILES	**DELIVERY**	**TYPE**
Multiple	03/29/2026	Standard Audit

4	0	2	1	1	0	4
Total Findings	Critical	Major	Medium	Minor	Informational	Resolved

🟥 0 Critical	An exposure that can affect the contract functions in several events that can risk and disrupt the contract
🟧 2 Major	An opening & exposure to manipulate the contract in an unwanted manner
🟧 1 Medium	An opening that could affect the outcome in executing the contract in a specific situation
⬜ 1 Minor	An opening but doesn't have an impact on the functionality of the contract
🟦 0 Informational	An opening that consists information but will not risk or affect the contract
🟩 4 Resolved	ContractWolf's findings has been acknowledged & resolved by the project

STATUS ✔ **AUDIT PASSED**

TABLE OF CONTENTS | FUSDExchange V3

Summary

Project Summary
Findings Summary
Disclaimer
Scope of Work
Auditing Approach

Project Information

Token/Project Details
Inheritance Graph
Call Graph

Findings

Issues
SWC Attacks
CW Assessment
Fixes & Recommendation
Audit Comments

DISCLAIMER | FUSDExchange V3

ContractWolf audits and reports should not be considered as a form of project's "Advertisement" and does not cover any interaction and assessment from "Project Contract" to "External Contracts" such as PancakeSwap, UniSwap, SushiSwap or similar.

ContractWolf does not provide any warranty on its released report and should not be used as a decision to invest into audited projects.

ContractWolf provides a transparent report to all its "Clients" and to its "Clients Participants" and will not claim any guarantee of bug-free code within its **SMART CONTRACT**.

ContractWolf's presence is to analyze, audit and assess the Client's Smart Contract to find any underlying risk and to eliminate any logic and flow errors within its code.

Each company or project should be liable to its security flaws and functionalities.

SCOPE OF WORK | FUSDExchange V3

FUSD's team has agreed and provided us with the files that need to be tested (*Github, BSCscan, Etherscan, Local files etc*). The scope of audit is the main contract.

The goal of this engagement is to identify if there is a possibility of security flaws in the implementation of smart contract and its systems.

ContractWolf will be focusing on contract issues and functionalities along with the project claims from smart contract to their website, whitepaper, repository which has been provided by **FUSD**.

AUDITING APPROACH | FUSDExchange V3

Every line of code along with its functionalities will undergo manual review to check for security issues, quality of logic and contract scope of inheritance. The manual review will be done by our team that will document any issues that they discovered.

METHODOLOGY

The auditing process follows a routine series of steps :

1. Code review that includes the following :
   - Review of the specifications, sources and instructions provided to ContractWolf to make sure we understand the size, scope and functionality of the smart contract.
   - Manual review of code. Our team will have a process of reading the code line-by-line with the intention of identifying potential vulnerabilities, underlying and hidden security flaws.

2. Testing and automated analysis that includes :
   - Testing the smart contract function with common test cases and scenarios to ensure that it returns the expected results.

3. Best practices and ethical review. The team will review the contract with the aim to improve efficiency, effectiveness, clarifications, maintainability, security and control within the smart contract.

4. Recommendations to help the project take steps to eliminate or minimize threats and secure the smart contract.

TOKEN DETAILS | FUSDExchange V3

The World's Leading Appreciating Stable Coin

Token Name	Symbol	Decimal	Total Supply	Chain
-	-	-	-	BESC, ETH, BSC, MATIC

SOURCE

Source	*FUSDExchangeV3 : 0x2917A24D7491C99b9eB453F095b3a447366D3fb0* *FUSDTreasury : 0xC94b125cC2a3BDBB12af8Ae5D415Ee856a1e756A* *vUSD : 0xc1A1EA08EEF9b64E1FFF2b8c17e3F2255E7fa118*

FINDINGS | FUSDExchange V3

Total Findings	Critical	Major	Medium	Minor	Informational	Resolved
4	0	2	1	1	0	4

This report has been prepared to state the issues and vulnerabilities for FUSD through this audit. The goal of this report findings is to identify specifically and fix any underlying issues and errors

ID	Title	File & Line #	Severity	Status
	Treasury Sync Bug	Treasury, Exchange	Major	Resolved
SWC-114	Transaction Order Dependence	Exchange	Major	Acknowledged
	Hardcoded Peg Assumption in Swaps	Exchange	Medium	Resolved
SWC-112	DoS with Unexpected Revert	Exchange	Minor	Resolved

SWC ATTACKS | FUSDExchange V3

Smart Contract Weakness Classification and Test Cases

ID	Description	Status
SWC-100	Function Default Visibility	Passed
SWC-101	Integer Overflow and Underflow	Passed
SWC-102	Outdated Compiler Version	Passed
SWC-103	Floating Pragma	Passed
SWC-104	Unchecked Call Return Value	Passed
SWC-105	Unprotected Ether Withdrawal	Passed
SWC-106	Unprotected SELF DESTRUCT Instruction	Passed
SWC-107	Reentrancy	Passed
SWC-108	State Variable Default Visibility	Passed
SWC-109	Uninitialized Storage Pointer	Passed
SWC-110	Assert Violation	Passed
SWC-111	Use of Deprecated Solidity Functions	Passed
SWC-112	Delegate call to Untrusted Callee	Passed
SWC-113	DoS with Failed Call	Passed
SWC-114	Transaction Order Dependence	Passed
SWC-115	Authorization through tx.origin	Passed
SWC-116	Block values as a proxy for time	Passed
SWC-117	Signature Malleability	Passed
SWC-118	Incorrect Constructor Name	Passed
SWC-119	Shadowing State Variables	Passed
SWC-120	Weak Sources of Randomness from Chain Attributes	Passed
SWC-121	Missing Protection against Signature Replay Attacks	Passed
SWC-122	Lack of Proper Signature Verification	Passed

ID	Description	Status
SWC-123	Requirement Violation	Passed
SWC-124	Write to Arbitrary Storage Location	Passed
SWC-125	Incorrect Inheritance Order	Passed
SWC-126	Insufficient Gas Griefing	Passed
SWC-127	Arbitrary Jump with Function Type Variable	Passed
SWC-128	DoS With Block Gas Limit	Passed
SWC-129	Typographical Error	Passed
SWC-130	Right-To-Left-Override control character(U+202E)	Passed
SWC-131	Presence of unused variables	Passed
SWC-132	Unexpected Ether balance	Passed
SWC-133	Hash Collisions With Multiple Variable Arguments	Passed
SWC-134	Message call with hardcoded gas amount	Passed
SWC-135	Code With No Effects	Passed
SWC-136	Unencrypted Private Data On-Chain	Passed

CW ASSESSMENT | FUSDExchange V3

ContractWolf Vulnerability and Security Tests

ID	Name	Description	Status
CW-001	Multiple Version	Presence of multiple compiler version across all contracts	✔
CW-002	Incorrect Access Control	Additional checks for critical logic and flow	✔
CW-003	Payable Contract	A function to withdraw ether should exist otherwise the ether will be trapped	✔
CW-004	Custom Modifier	major recheck for custom modifier logic	✔
CW-005	Divide Before Multiply	Performing multiplication before division is generally better to avoid loss of precision	✔
CW-006	Multiple Calls	Functions with multiple internal calls	✔
CW-007	Deprecated Keywords	Use of deprecated functions/operators such as block.blockhash() for blockhash(), msg.gas for gasleft(), throw for revert(), sha3() for keccak256(), callcode() for delegatecall(), suicide() for selfdestruct(), constant for view or var for actual type name should be avoided to prevent unintended errors with newer compiler versions	✔
CW-008	Unused Contract	Presence of an unused, unimported or uncalled contract	✔
CW-009	Assembly Usage	Use of EVM assembly is error-prone and should be avoided or double-checked for correctness	✔
CW-010	Similar Variable Names	Variables with similar names could be confused for each other and therefore should be avoided	✔
CW-011	Commented Code	Removal of commented/unused code lines	✔
CW-012	SafeMath Override	SafeMath is no longer needed starting with Solidity v0.8+. The compiler now has built-in overflow checking.	✔

FIXES & RECOMMENDATION

Treasury Sync Bug | RESOLVED

Affected : ***FUSDTreasury***, ***FUSDExchangeV3***

Instead of a manual counter **`totalLiquidityUSDC`**, the exchange should query the actual balance of the treasury.

Recommendation

Check the actual balance

```
// In FUSDExchangeV3.sol
function getActualLiquidity() public view returns (uint256) {
    return usdcToken.balanceOf(address(treasury));
}

// Update sellFUSD to check actual balance
function sellFUSD(uint256 fusdAmount) external nonReentrant tradingAllowed {
    //
    if (usdcAmount > getActualLiquidity()) revert InsufficientLiquidity();
    //
}
```

Hardcoded Peg Assumption in Swaps | ACKNOWLEDGED

In **`buyFUSDWithStable`**, the **`expectedUsdcOut`** is calculated assuming the stablecoin is perfectly pegged **1:1** to USDC

If a supported stablecoin like USDC depegs even slightly, the minAmountOut (slippage) calculation will be inaccurate,

leading to reverted transactions or vulnerability to MEV sandwiches during the swap.

Recommendation

Use a live price-feed (ignore if intentional)

SWC-114 | Transaction Ordering Dependence | RESOLVED

Affected : ***buyFUSD***, ***sellFUSD***, ***buyFUSDWithStable***, ***sellFUSDForStable***

None of the trade functions provide slippage protection.

The amount of **FUSD** a user receives is calculated using the current price at execution time. An attacker can front-run the transaction, alter the price and cause the user to receive a worse rate than expected.

Recommendation

Add `minFusdAmount` / `maxFusdAmount` parameters to all trade functions.

Validate that the executed amount is within the user's acceptable range.

```
function buyFUSD(uint256 usdcAmount, uint256 minFusdAmount) external
nonReentrant tradingAllowed {
     //
    uint256 fusdAmount = (netUsdcAmount * decimalsAdjustment * 1e18) / price;
    require(fusdAmount >= minFusdAmount, "Slippage too high");
    //
}
```

SWC-112 | DoS with Unexpected Revert | RESOLVED

The **`_recordLiquidityFee`** function uses **`try/catch`** to call **`recordFee`** on the liquidityWallet. If **`liquidityWallet`** is an **EOA** (no code), the call is skipped silently.

While this prevents a revert, it also means fee recording may fail without any indication. This could lead to accounting mismatches if fees are expected to be tracked.

Recommendation

```
function _recordLiquidityFee(uint256 amount) internal {
    if (amount == 0) return;
    // Option 1: require the wallet to be a contract
    require(liquidityWallet.code.length > 0, "Invalid fee wallet");
    ILiquidityFeeVault(liquidityWallet).recordFee(amount);
    // Option 2: keep try/catch but emit an event on failure
    // try ILiquidityFeeVault(liquidityWallet).recordFee(amount) {} catch {
    //     emit FeeRecordingFailed(amount);
    // }
}
```

AUDIT COMMENTS | FUSDExchange V3

Smart Contract audit comment for a non-technical perspective

- 🟢 Owner can set taxes
- 🟢 Contract can be paused / Emergency Stop
- 🟢 Owner can renounce and transfer ownership
- 🟢 Owner can burn tokens
- 🟢 Owner cannot mint after initial deployment
- 🟢 Owner cannot set max transaction limit
- 🟢 Owner cannot block users

CONTRACTWOLF

Blockchain Security - Smart Contract Audits